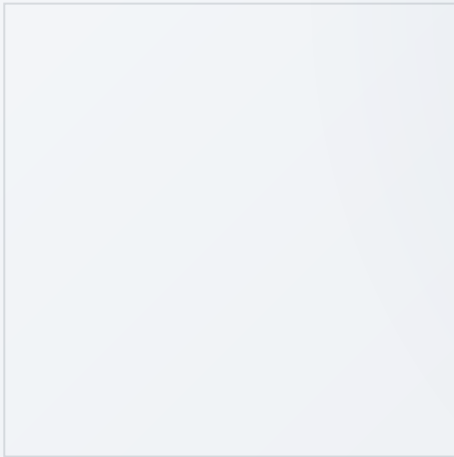




Searching & Hashing

Complete Algorithm Reference Notes

Linear Search • Binary Search • Hash Tables

Hash Functions • Collision Resolution • Open Addressing

Comprehensive Analysis with Proofs, Complexity Derivations & Visual Walkthroughs



Table of Contents

Part 1: Linear Search

- 1.1 Algorithm Overview
- 1.2 Complete Complexity Analysis
- 1.3 Properties
- 1.4 Visual Walkthrough
- 1.5 Loop Invariant & Correctness Proof

Part 2: Binary Search

- 2.1 Algorithm Overview
- 2.2 Complete Complexity Analysis
- 2.3 Properties
- 2.4 Visual Walkthrough
- 2.5 Loop Invariant & Correctness Proof

Part 3: Hash Tables

- 3.1 Introduction & Direct Addressing Problem
- 3.2 Hash Table Core Concept
- 3.3 The Collision Problem
- 3.4 Independent Uniform Hashing
- 3.5 Collision Resolution by Chaining
- 3.6 Hash Table Operations & Analysis

Part 4: Hash Functions

- 4.1 What Makes a Good Hash Function?
- 4.2 Static vs. Random Hashing
- 4.3 The Division Method
- 4.4 The Multiplication Method
- 4.5 The Multiply-Shift Method

Part 5: Open Addressing

5.1 Concept & Overview

5.2 Linear Probing

5.3 Quadratic Probing

5.4 Double Hashing

5.5 Deletion in Open Addressing

5.6 Comparison of Probing Methods

Part 6: Summary & Comparison Tables

Part 1: Linear Search

1.1 Algorithm Overview

Type: Sequential / Brute-force search algorithm.

How it works: Linear Search examines each element of the array sequentially from the first index to the last, comparing every element with the target key until either a match is found or the entire array has been traversed.

Algorithm: Linear-Search($A[1\dots n]$, key)

```
1  i ← 1
2  while i ≤ n do
3      if A[i] = key then
4          return i
5      end
6      i ← i + 1
7  end
8  return 0          // key not found
```

Best cases to use this algorithm:

- When the array is **unsorted** or **unorganized**.
- When the dataset is **small** (small n).
- When searching is performed **infrequently**.
- When the **target is expected to be near the beginning** of the array.
- When memory access patterns are sequential and cache-friendly for small arrays.

1.2 Complete Complexity Analysis

1.2.1 RAM Model & Line-by-Line Cost

RAM Model Assumptions:

- Each primitive operation (assignment, comparison, arithmetic) takes constant time.
- Memory access is uniform and constant time.

Table 1: Line-by-line cost analysis for Linear Search

Line	Operation	Cost	Times (Worst Case)
1	$i \leftarrow 1$	c_1	1
2	$i \leq n$ (loop test)	c_2	$n + 1$
3	$A[i] = \text{key}$ (compare)	c_3	n
4	$\text{return } i$	c_4	0 (worst case)
6	$i \leftarrow i + 1$	c_5	n
8	$\text{return } 0$	c_6	1

1.2.2 Recurrence Relation & Full Derivation

Let $T(n)$ be the total number of comparisons (or time) for an array of size n .

Recurrence for Worst Case (unsuccessful search):

$$T(0) = 0 \quad (\text{empty array, no comparisons})$$

$$T(n) = T(n - 1) + 1 \quad \text{for } n > 0$$

Step-by-step solving by substitution (expansion method):

We expand the recurrence by repeatedly substituting the definition of T :

$$\begin{aligned}
 T(n) &= T(n - 1) + 1 \\
 &= [T(n - 2) + 1] + 1 = T(n - 2) + 2 \\
 &= [T(n - 3) + 1] + 2 = T(n - 3) + 3 \\
 &\vdots \\
 &= T(n - k) + k
 \end{aligned}$$

We reach the base case when $n - k = 0$, i.e., $k = n$:

$$T(n) = T(0) + n = 0 + n = n$$

Result:

$$T(n) = n \Rightarrow T(n) = \Theta(n)$$

1.2.3 Alternative Derivation: Summing the Loop

The total worst-case cost can also be obtained by direct summation:

$$\begin{aligned} T(n) &= c_1 + (n + 1) \cdot c_2 + n \cdot c_3 + n \cdot c_5 + c_6 \\ &= (c_2 + c_3 + c_5) \cdot n + (c_1 + c_2 + c_6) \\ &= a \cdot n + b \quad (\text{a linear function of } n) \\ &= \Theta(n) \end{aligned}$$

1.2.4 All Cases Summary**Table 2: Linear Search: complexity cases**

Case	Scenario	# Comparisons	Complexity
Best Case	Key at $A[1]$ (first position)	1	$\Theta(1)$
Average Case	Key at random position or not present	$\frac{n+1}{2}$	$\Theta(n)$
Worst Case	Key at $A[n]$ or not present	n	$\Theta(n)$

Average case detailed analysis: If the key is equally likely to be at any position, the expected number of comparisons is:

$$E[\text{comparisons}] = \frac{1}{n} \sum_{i=1}^n i = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2}$$

If the key is not present with probability p , then $E = p \cdot n + (1 - p) \cdot \frac{n+1}{2} = \Theta(n)$.

1.3 Properties**Table 3: Linear Search properties**

Property	Linear Search	Reason
----------	---------------	--------

Stable	N/A	Search algorithms do not reorder elements; stability (preservation of relative order of equal keys) is a property of sorting algorithms.
In-Place	Yes	Requires only $O(1)$ extra space: one index variable i . No additional data structures needed.
Adaptive	Partially	Terminates early if target is near the beginning. However, the algorithm always starts from index 1 regardless of input patterns.

1.4 Visual Walkthrough

Array A: [7, 12, 5, 22, 13, 32] (indices 1--6) Target key: 13

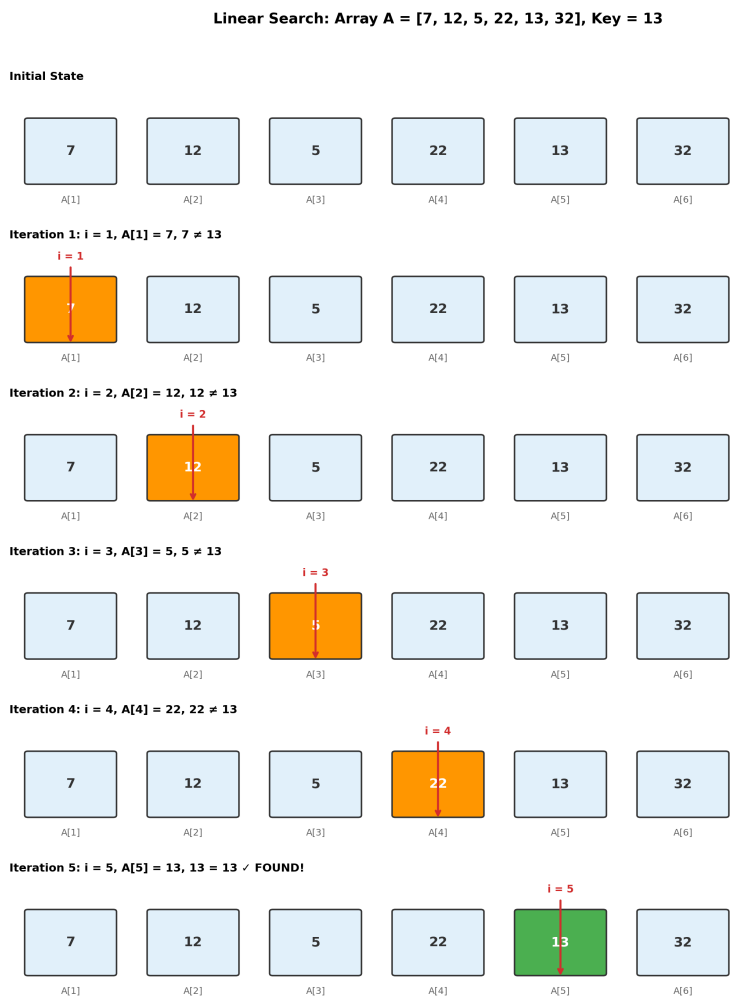


Figure 1: Linear Search iteration steps: checking each element sequentially until key 13 is found at index 5.

Table 4: Linear Search step-by-step trace

4th	4	22	$22 \neq 13$	$i \leftarrow 5$
5th	5	13	$13 = 13$	return 5

1.5 Loop Invariant & Correctness Proof

1.5.1 Loop Invariant

Hypothesis $H(r)$:

When the control reaches the "while" statement for the r -th time:

1. $i = r$
2. $key \notin A[1 \dots r - 1]$ (key is not present in the already-examined subarray)

Range: $\forall 1 \leq r \leq n + 1$

1.5.2 Proof by Mathematical Induction

Base Case ($r = 1$):

- Before the first iteration: $i = 1$ ✓
- $A[1 \dots 0]$ is empty, so $key \notin A[1 \dots 0]$ holds **vacuously** ✓

Induction Hypothesis: Assume $H(m)$ holds for some m where $1 \leq m \leq n$:

- $i = m$
- $key \notin A[1 \dots m - 1]$

Inductive Step (prove $H(m + 1)$):

At the start of iteration m , we check $A[m]$:

- **Case A:** If $A[m] = key$, the algorithm returns m (correct index). Loop terminates.
- **Case B:** If $A[m] \neq key$, then $key \notin A[1 \dots m]$ (since it was not in $A[1 \dots m - 1]$ by I.H., and $A[m] \neq key$). Then i increments to $m + 1$.

Therefore, at the start of the next iteration ($r = m + 1$): $i = m + 1$ ✓ and $key \notin A[1 \dots m]$ ✓

Thus $H(m) \Rightarrow H(m + 1)$. By induction, $H(r)$ holds for all valid r .

1.5.3 Using the Invariant to Prove Correctness

Termination Analysis: The loop terminates when either:

1. $A[i] = \text{key}$ is found $\rightarrow$ returns correct index i .
2. $i > n \rightarrow$ loop exits, returns 0.

Case 1 (Key found at position $k \leq n$):

By $H(k)$: $i = k$ and $\text{key} \notin A[1 \dots k - 1]$. When $A[k] = \text{key}$, return k -- this is the **first occurrence** ✓

Case 2 (Key not in array):

Loop runs until $i = n + 1$. By $H(n + 1)$: $i = n + 1$ and $\text{key} \notin A[1 \dots n]$. Loop condition $i \leq n$ fails, return 0 -- correct "not found" signal ✓

Conclusion:

The Linear Search algorithm is correct.

blacksquare

Part 2: Binary Search

2.1 Algorithm Overview

Type: Divide-and-Conquer search algorithm.

Precondition: The input array must be **sorted** (monotonically increasing or decreasing).

How it works:

1. Compare the target key with the middle element of the current interval.
2. If key equals middle element, return the middle index.
3. If key is less than middle element, search the left half.
4. If key is greater than middle element, search the right half.
5. Repeat until found or interval is empty.

Algorithm: Binary-Search-Recursive(A, first, last, key)

```
1  if first ≤ last then
2      mid ← ⌊(first + last) / 2⌋
3      if A[mid] = key then
4          return mid
5      end
6      if A[mid] < key then
7          return Binary-Search-Recursive(A, mid+1, last, key)
8      end
9      else
10         return Binary-Search-Recursive(A, first, mid-1, key)
11     end
12 end
13 else
14     return -1
15 end
```

Best cases to use this algorithm:

- When the array is **sorted** (precondition).
- When the dataset is **large** (large n).
- When **frequent searches** are performed on the same dataset (amortize sorting cost).
- When **random access** is available (arrays, not linked lists).

- When $O(\log n)$ time complexity is required.

2.2 Complete Complexity Analysis

2.2.1 RAM Model & Line-by-Line Cost

Table 5: Line-by-line cost analysis for Binary Search

Line	Operation	Cost
1	$first \leq last$ (comparison)	c_1
2	$mid \leftarrow \lfloor (first + last)/2 \rfloor$	c_2
3	$A[mid] = key$ (compare)	c_3
4	$return\ mid$	c_4
6	$A[mid] < key$ (compare)	c_5
7, 10	Recursive call	$T(n/2)$
14	$return\ -1$	c_8

2.2.2 Recurrence Relation & Full Derivation

Let $W(n)$ be the number of comparisons in the worst case for an array of size n .

Recurrence for Worst Case:

$$W(0) = 1 \quad (\text{one comparison: } first \leq last \text{ fails})$$

$$W(1) = 2 \quad (first \leq last \text{ check} + A[mid] = key \text{ check})$$

$$W(n) = W(\lfloor n/2 \rfloor) + 2 \quad \text{for } n > 1$$

Method 1: Substitution (Expansion)

$$\begin{aligned}
W(n) &= W(n/2) + 2 \\
&= [W(n/4) + 2] + 2 = W(n/4) + 2 \cdot 2 \\
&= [W(n/8) + 2] + 4 = W(n/8) + 3 \cdot 2 \\
&\vdots \\
&= W(n/2^k) + 2k
\end{aligned}$$

Base case reached when $n/2^k = 1$, i.e., $2^k = n$, so $k = \log_2 n$:

$$\begin{aligned}
W(n) &= W(1) + 2 \log_2 n \\
&= 2 + 2 \log_2 n \\
&= 2 \log_2 n + 2 \\
&= \Theta(\log n)
\end{aligned}$$

Method 2: Master Theorem

The recurrence $T(n) = T(n/2) + O(1)$ fits the Master Theorem form $T(n) = aT(n/b) + f(n)$ with $a = 1, b = 2, f(n) = O(1) = O(n^0)$.

Compare $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$ with $f(n) = 1$.

Since $f(n) = \Theta(n^{\log_b a})$, we are in **Case 2** of the Master Theorem:

$$T(n) = \Theta(n^{\log_b a} \log n) = \Theta(\log n)$$

Method 3: Alternative (counting only key comparisons)

If we count only the comparison $A[mid] \stackrel{?}{=} key$:

$$T(n) = T(n/2) + 1, \quad T(1) = 1$$

Expanding:

$$\begin{aligned}
T(n) &= T(n/2) + 1 \\
&= T(n/4) + 1 + 1 = T(n/4) + 2 \\
&= T(n/8) + 3 \\
&\vdots \\
&= T(n/2^k) + k
\end{aligned}$$

Set $n/2^k = 1 \Rightarrow k = \log_2 n$:

$$T(n) = T(1) + \log_2 n = 1 + \log_2 n = \Theta(\log n)$$

2.2.3 All Cases Summary

Table 6: Binary Search: complexity cases

Case	Scenario	# Comparisons	Complexity
Best Case	Key is at the middle (first check)	2	$\Theta(1)$
Average Case	Key at random position or not present	$\sim \log_2 n$	$\Theta(\log n)$
Worst Case	Key not present, or at first/last position	$2 \log_2 n + 2$	$\Theta(\log n)$

2.3 Properties

Table 7: Binary Search properties

Property	Binary Search	Reason
Stable	N/A	Binary search is read-only; it never reorders or modifies the array.
In-Place	Yes	Requires only $O(1)$ extra space: variables <i>first</i> , <i>last</i> , <i>mid</i> . Even the recursive version uses $O(\log n)$ stack space, which is considered in-place for search algorithms.
Adaptive	No	Always divides the array in half regardless of input distribution. The split point is always the middle.

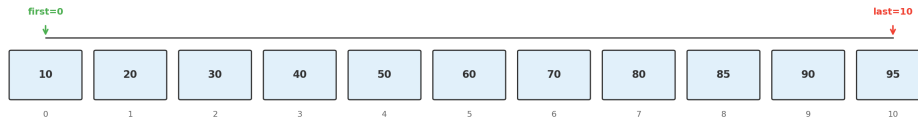
2.4 Visual Walkthrough

Array *A* (sorted): [10, 20, 30, 40, 50, 60, 70, 80, 85, 90, 95] (indices 0–10) Target key: 65

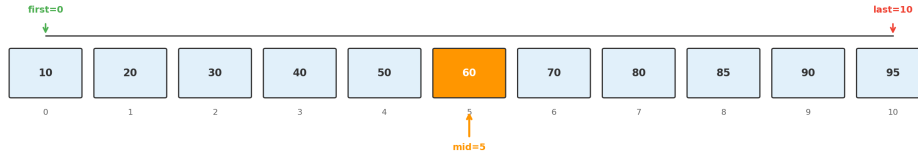
Part 2: Binary Search

Binary Search: Array A = [10, 20, 30, 40, 50, 60, 70, 80, 85, 90, 95], Key = 65

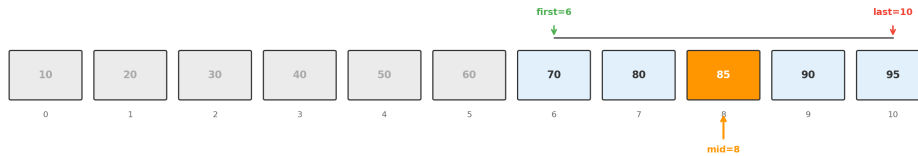
Initial: first = 0, last = 10, Search entire array



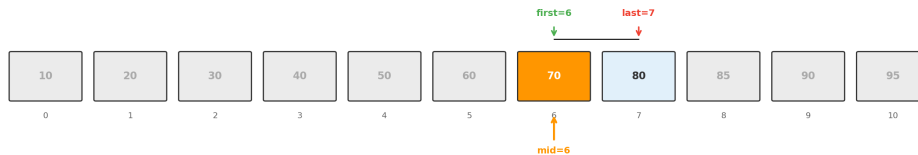
Iteration 1: mid = 5, A[5] = 60 < 65 → Search RIGHT: first = 6



Iteration 2: mid = 8, A[8] = 85 > 65 → Search LEFT: last = 7



Iteration 3: mid = 6, A[6] = 70 > 65 → Search LEFT: last = 5



Iteration 4: first = 6 > last = 5 → STOP, return -1 (Not Found)



Figure 2: Binary Search iteration steps searching for key 65 (not found). Each iteration halves the search interval.

Table 8: Binary Search trace: key = 65 (unsuccessful)

Iteration	<i>first</i>	<i>last</i>	<i>mid</i>	$A[mid]$	Comparison	Action
Initial	0	10	--	--	--	--
1st	0	10	5	60	$60 < 65$	$first \leftarrow 6$
2nd	6	10	8	85	$85 > 65$	$last \leftarrow 7$
3rd	6	7	6	70	$70 > 65$	$last \leftarrow 5$
4th	6	5	--	--	$6 > 5$	return -1

Example where key IS found (key = 70):

Table 9: Binary Search trace: key = 70 (successful)

2.5 Loop Invariant & Correctness Proof

2.5.1 Loop Invariant

Invariant I :

At the start of each recursive call `Binary-Search-Recursive( $A$ ,  $first$ ,  $last$ ,  $key$ )`:

If key is present in array A at any index p , then $first \leq p \leq last$.

Equivalently: The subarray $A[first \dots last]$ contains all possible locations where key could be found.

2.5.2 Proof by Mathematical Induction

Initialization (Base Case):

- Initial call: $first = 0$, $last = n - 1$ (entire array).
- If key is anywhere in A , it is certainly in $A[0 \dots n - 1]$ ✓
- **Invariant holds.**

Maintenance (Inductive Step):

Assume I holds at the start of a call with parameters $(first, last)$. Let $mid = \lfloor (first + last) / 2 \rfloor$.

- **Case 1:** $A[mid] = key \rightarrow$ return mid . Correct! ✓
- **Case 2:** $A[mid] < key$ -- Since the array is sorted, key cannot be in $A[first \dots mid]$ (all elements $\leq A[mid] < key$). If key exists, it must be in $A[mid + 1 \dots last]$. Recursive call: $(mid + 1, last)$ -- invariant maintained ✓
- **Case 3:** $A[mid] > key$ -- Since the array is sorted, key cannot be in $A[mid \dots last]$ (all elements $\geq A[mid] > key$). If key exists, it must be in $A[first \dots mid - 1]$. Recursive call: $(first, mid - 1)$ -- invariant maintained ✓

In all cases, if key exists, the invariant is preserved.

Termination:

- The interval size strictly decreases: $last - first + 1$ becomes at most $\lceil n/2 \rceil$.
- Eventually $first > last$ (empty interval).
- At this point, by the contrapositive of the invariant: if the interval is empty and we have not found key , then key is **not present** in the original array.

- Return -1 is correct ✓

2.5.3 Correctness Proof Using the Invariant

Theorem: Binary Search returns the index mid where $A[mid] = key$ if key exists in A , and -1 if key does not exist in A .

Soundness (if algorithm returns index, it is correct):

The algorithm only returns mid after checking $A[mid] = key$ (line 3). Therefore the returned index is always correct ✓

Completeness (if key exists, algorithm finds it):

By the invariant, if key is in A , it is always within the current search interval. The interval size decreases by at least half each step: $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$. After at most $\lceil \log_2(n + 1) \rceil$ steps, interval size becomes 1. At that point, $mid = first = last$, and we check if $A[mid] = key$. If yes, return it. If no, the subsequent call will have $first > last$, and by the invariant contrapositive, key was never in the array ✓

Conclusion:

Binary Search is correct. ■

Part 3: Hash Tables

3.1 Introduction & The Problem with Direct Addressing

A **direct-address table** is an array $T[0 \dots m - 1]$ where an element with key k is stored directly in slot $T[k]$. This provides $O(1)$ worst-case time for all dictionary operations (INSERT, SEARCH, DELETE).

The Downside of Direct Addressing:

If the universe U of possible keys is **large** or **infinite**, storing a table T of size $|U|$ becomes **impractical** or **impossible** given typical memory constraints. The set K of keys actually stored may be very small relative to U , causing most of the space allocated for T to be wasted.

Key Insight: When $|K| \ll |U|$ (the set of stored keys is much smaller than the universe of all possible keys), a hash table provides a much more storage-efficient solution.

3.2 Hash Table Core Concept

A **hash table** reduces the storage requirement to $\Theta(|K|)$ while maintaining $O(1)$ *average-case* search time.

Table 10: Direct Addressing vs. Hash Tables

Aspect	Direct Addressing	Hash Tables
Time Bound	Worst-case $O(1)$	Average-case $O(1)$
Storage	$\Theta(U)$	$\Theta(K)$
Slot Determination	Element with key k stored in slot k	Element with key k stored in slot $h(k)$

Hash Function Definition

$$h : U \rightarrow \{0, 1, \dots, m - 1\}$$

Where:

- U = universe of all possible keys.
- m = size of hash table (typically $m \ll |U|$).
- $h(k)$ = hash value of key k (the slot number where key k is stored).

Terminology:

- An element with key k **hashes to** slot $h(k)$.
- $h(k)$ is the **hash value** of key k .

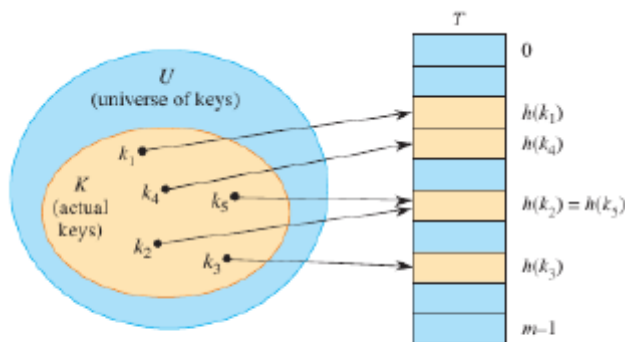


Figure 11.2 Using a hash function h to map keys to hash-table slots. Because keys k_2 and k_3 map to the same slot, they collide.

Figure 3: Using a hash function h to map keys to hash-table slots. Because keys k_2 and k_3 map to the same slot, they collide.

3.3 The Collision Problem

Definition: Collision

A **collision** occurs when two distinct keys $k_1 \neq k_2$ hash to the same slot: $h(k_1) = h(k_2)$.

Fundamental Constraint: Since $|U| > m$, collisions are **unavoidable** by the pigeonhole principle -- there must exist at least two keys with the same hash value.

Ideal vs. Reality:

- **Ideal:** Avoid collisions altogether.
- **Reality:** Impossible to avoid completely when $|U| > m$.
- **Practical Goal:** Use a well-designed hash function to **minimize** collisions, plus a **collision resolution technique**.

Important Note:

A hash function h must be **deterministic** -- a given input k must always produce the same output $h(k)$.

3.4 Independent Uniform Hashing

Definition: Independent Uniform Hashing (SUHA)

Each key is equally likely to hash to any of the m slots, independently of where any other keys have hashed to. Formally:

1. **Uniformity:** $\Pr[h(k) = i] = \frac{1}{m}$ for all $i \in \{0, 1, \dots, m - 1\}$.
2. **Independence:** The hash values of different keys are independent random variables.

Important caveats: In practice, truly uniform and independent hash functions do not exist for arbitrary key sets. SUHA is a theoretical model used to analyze expected performance. **Universal hashing** provides a practical way to approximate these assumptions by randomly selecting a hash function from a carefully designed family.

3.5 Collision Resolution by Chaining

Hashing with chaining is a **nonrecursive form of divide-and-conquer**:

- The input set of n elements is divided randomly into m subsets.
- Each subset has approximate size n/m .
- The hash function determines which subset an element belongs to.
- Each subset is managed independently as a **linked list**.

Structure

- Each nonempty slot points to a **linked list**.
- All elements that hash to the same slot go into that slot's linked list.
- Slot j contains a pointer to the **head** of the list of all stored elements with hash value j .
- If no elements hash to slot j , then slot j contains **NIL**.

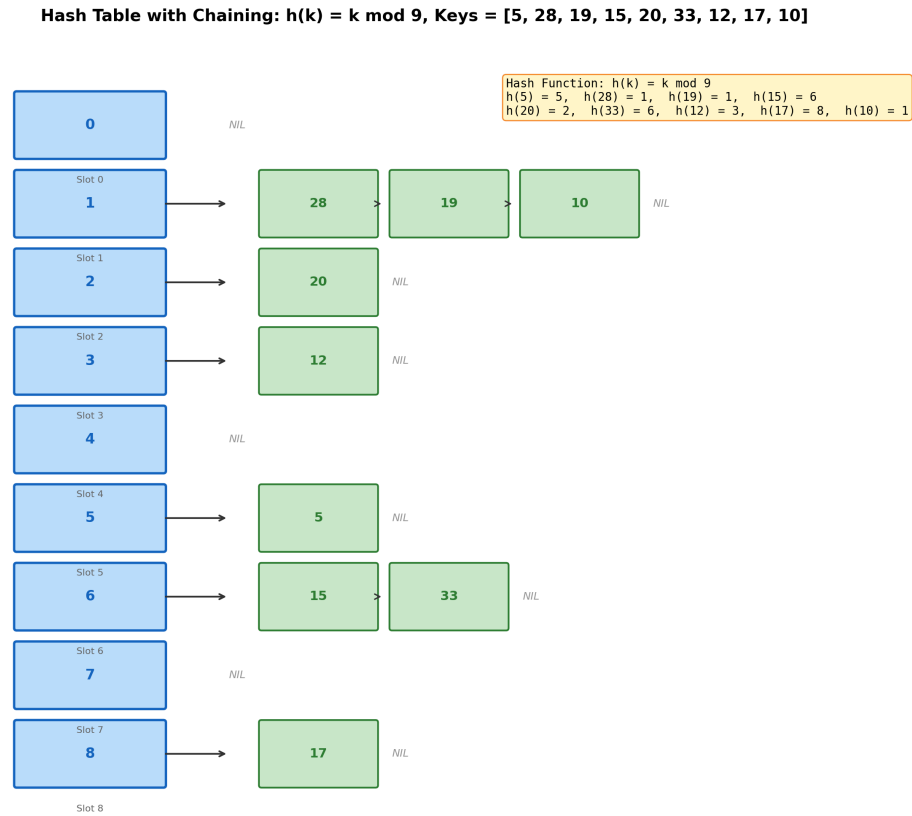


Figure 4: Hash table with chaining. Each slot points to a linked list of all keys that hash to it. Keys that collide are stored in the same linked list.

Dictionary Operations

Table 11: Chaining: dictionary operations

Operation	Pseudocode	Worst-Case Time	Expected Time (SUHA)
INSERT	LIST-PREPEND($T[h(x.key)], x$)	$O(1)$	$O(1)$
SEARCH	LIST-SEARCH($T[h(k)], k$)	$O(n)$	$O(1 + \alpha)$
DELETE	LIST-DELETE($T[h(x.key)], x$)	$O(1)$	$O(1)$

Deletion details:

- **Doubly linked lists:** $O(1)$ time (input is element x , not key k , so no search needed).
- **Singly linked lists:** Time proportional to list length.
- **Recommendation:** If the hash table supports deletion, linked lists should be **doubly linked**.

Expected Time Analysis Under SUHA

traverse the entire list at $h(k)$:

$$E[\text{unsuccessful search time}] = \Theta(1 + \alpha)$$

Analysis of successful search: Consider the i -th inserted element x_i . At the time of insertion, there are $i - 1$ elements already in the table. The expected length of the list where x_i is inserted is $(i - 1)/m$. The expected number of elements examined in a successful search for x_i is $1 + (i - 1)/m$ (1 for x_i itself, plus the expected number of elements inserted after x_i that hash to the same slot... actually, we need to average over all n elements):

$$\begin{aligned} E[\text{successful search time}] &= \frac{1}{n} \sum_{i=1}^n \left(1 + \frac{i-1}{m} \right) \\ &= 1 + \frac{1}{nm} \sum_{i=1}^n (i-1) \\ &= 1 + \frac{1}{nm} \cdot \frac{(n-1)n}{2} \\ &= 1 + \frac{n-1}{2m} \\ &= 1 + \frac{\alpha}{2} - \frac{1}{2m} \\ &= \Theta(1 + \alpha) \end{aligned}$$

Result:

Under SUHA, chaining achieves $\Theta(1 + \alpha)$ expected time for both successful and unsuccessful search. When $\alpha = O(1)$ (i.e., $n = O(m)$), all operations take $O(1)$ **expected time**.

3.6 Hash Table Operations & Pseudocode

CHAINED-HASH-INSERT(T, x)

```
1 LIST-PREPEND(T[h(x.key)], x)
```

CHAINED-HASH-SEARCH(T, k)

```
1 return LIST-SEARCH(T[h(k)], k)
```

CHAINED-HASH-DELETE(T, x)

```
1  LIST-DELETE( $T[h(x.key)], x$ )
```

===== PART 4: HASH FUNCTIONS ===== -->

Part 4: Hash Functions

4.1 What Makes a Good Hash Function?

A good hash function must satisfy two key requirements:

1. **Efficiently computable:** The hash value should be computed quickly, ideally in $O(1)$ time.
2. **Satisfies (approximately) independent uniform hashing:** Each key is equally likely to hash to any of the m slots, independently of where any other keys have hashed to.

The Problem with "Equally Likely": If the hash function is fixed, probabilities depend on the **probability distribution of input keys**. Typically, you have no way to verify this condition unless you know the key distribution. Keys might not be drawn independently.

Known distribution example: If keys are random real numbers k , independently and uniformly distributed in range $0 \leq k < 1$, then $h(k) = \lfloor km \rfloor$ satisfies independent uniform hashing.

4.2 Static vs. Random Hashing

Table 12: Static Hashing vs. Random Hashing

Aspect	Static Hashing	Random Hashing
Definition	Hash function is fixed and does not change with the input.	Hash function is chosen randomly from a family of hash functions at runtime.
Universality	Not universal; may not evenly distribute keys.	Universal hash families guarantee low collision probability $\leq 1/m$.
Performance	Deterministic; may perform poorly on certain key sets.	Probabilistic guarantees; good average performance on any data.
Use Case	Simple implementations, known key distributions.	Robust against adversarial inputs, theoretical analysis.
Examples	Division method, Multiplication method.	Universal hash families, Multiply-Shift method.

Key Insight:

Random hashing provides provably good average-case performance for **any data** without knowing the input distribution. The randomization comes from the **known** process of picking the hash function, not the **unknown** process creating the input keys. **Universal hashing** is a particular kind of random hashing that works well.

Key Types in Practice

Hash functions handle keys of two types:

- **Type 1:** Short nonnegative integers -- fits in a w -bit machine word (typical $w = 32$ or 64).
- **Type 2:** Short vector of nonnegative integers (e.g., byte strings of variable length).

4.3 The Division Method**Formula:**

$$h(k) = k \bmod m$$

How it works: Computes the remainder when key k is divided by table size m .

Example: If $m = 12$ and $k = 100$, then $h(k) = 100 \bmod 12 = 4$.

Table 13: Division Method characteristics

Feature	Description
Speed	Very fast -- requires only a single division operation.
Constraint	Works best when m is a prime number not too close to an exact power of 2 .
Limitation	No guarantee of good average-case performance; constrains hash table size to be prime.

Critical Choice: Avoid powers of 2!

If $m = 2^p$, then $h(k) = k \bmod 2^p$ depends **only on the lowest p bits** of k , ignoring all higher bits. This causes many collisions when keys share the same low bits. **Best practice:** Choose m to be a prime number not too close to a power of 2.

4.4 The Multiplication Method

General Formula (Two Steps):

1. Multiply key k by constant A (where $0 < A < 1$) and extract the **fractional part** of kA .
2. Multiply this value by m and take the **floor** of the result.

$$h(k) = \lfloor m \cdot (kA \bmod 1) \rfloor$$

where $kA \bmod 1$ means the fractional part of kA , i.e., $kA - \lfloor kA \rfloor$.

Advantages:

- Value of m is **not critical** -- can choose m independently of how you choose A .
- Spreads keys more uniformly across all slots.
- Provides good foundation for **random hashing** (nonstatic hashing).

Good choice of A : Knuth suggests $A \approx (\sqrt{5} - 1)/2 \approx 0.6180339887...$ (the golden ratio conjugate). This irrational number tends to spread keys uniformly.

Example: For $m = 8$, $A = 0.618$, $k = 31$:

$$kA = 31 \times 0.618 = 19.158$$

$$kA \bmod 1 = 0.158$$

$$h(31) = \lfloor 8 \times 0.158 \rfloor = \lfloor 1.264 \rfloor = 1$$

4.5 The Multiply-Shift Method

This is a highly efficient hash function based on bitwise operations.

Formula:

Treat key k and a random odd constant a as w -bit integers. Compute:

$$h(k) = (a \cdot k) \gg (w - p)$$

where $m = 2^p$ (table size is a power of 2), and $\gg$ denotes right bit shift.

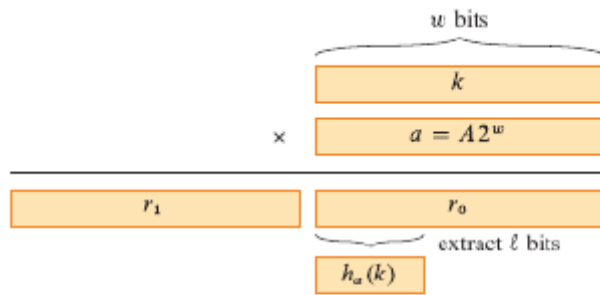


Figure 11.4 The multiply-shift method to compute a hash function. The w -bit representation of the key k is multiplied by the w -bit value $a = A \cdot 2^w$. The ℓ highest-order bits of the lower w -bit half of the product form the desired hash value $h_a(k)$.

Figure 5: The multiply-shift method: the w -bit representation of key k is multiplied by the w -bit value $a = A \cdot 2^w$. The ℓ highest-order bits of the lower w -bit half form the hash value $h_a(k)$.

How it works visually (from Figure 5):

1. Key k is represented as w bits.
2. Multiply k by w -bit value $a = A \cdot 2^w$.
3. The product produces two w -bit halves: r_1 (high) and r_0 (low).
4. Extract ℓ highest-order bits of the lower w -bit half (r_0).
5. These ℓ bits form the desired hash value $h_a(k)$.

Table 14: Multiply-Shift Method characteristics

Feature	Description
Speed	Extremely fast: single multiply + shift (2–3 CPU cycles on modern hardware).
Table size	Must be a power of 2 ($m = 2^p$).
Constant a	Must be odd (ensures multiplication is a bijection in w -bit space).
Theoretical guarantee	Part of 2-independent hash families.

Why must a be odd?

If a were even, then $a \cdot k$ would always be even, meaning the least significant bit would always be 0, severely limiting the hash distribution. An odd a ensures the multiplication is a bijection (permutation) in the w -bit space, preserving the randomness of key bits in the product.

Comparison of Hash Function Methods

Table 15: Comparison of hash function methods

Feature	Division	Multiplication	Multiply-Shift
Formula	$k \bmod m$	$\lfloor m(kA \bmod 1) \rfloor$	$(a \cdot k) \gg (w - p)$
Table Size	Best: prime	Any value	Must be power of 2
Speed	Fast	Moderate	Very fast
Key Dependency	All bits of k	All bits (via multiplication)	All bits (via multiplication)
Independence Level	None (static)	None (static)	2-independent

Part 5: Open Addressing

5.1 Concept & Overview

In **open addressing**, all elements are stored **directly in the hash table itself**. When a collision occurs, alternative slots within the table are probed until an empty slot is found.

Key difference from chaining:

- **Chaining:** Colliding keys are stored outside the table (in linked lists).
- **Open Addressing:** All keys are stored within the table; no external storage is used.

Probe sequence: For a key k , the hash function generates a sequence of slots to try:

$$(h(k, 0), h(k, 1), \dots, h(k, m - 1))$$

This sequence should be a **permutation** of $\{0, 1, \dots, m - 1\}$ to ensure all slots are eventually checked.

Table 16: Chaining vs. Open Addressing

Feature	Chaining	Open Addressing
Storage	Colliding keys stored in linked lists outside the table.	All keys stored within the table; alternative slots probed.
Extra Space	$O(n)$ for pointers and nodes.	No extra space beyond the table.
Load Factor	$\alpha = n/m$ can exceed 1 (unbounded).	Must have $\alpha \leq 1$ (cannot store more keys than slots).
Cache Performance	Poor: linked lists scatter memory accesses.	Better: all data in contiguous array.
Deletion	Simple: remove from linked list.	Complex: requires special handling (tombstones).
Clustering	No clustering within the table.	Primary and secondary clustering possible.
Expected Search	$O(1 + \alpha)$	$O\left(\frac{1}{1-\alpha}\right)$ for uniform probing

Open Addressing: Core Operations

HASH-INSERT(T, k)

```

1   $i \leftarrow 0$ 
2  repeat
3       $q \leftarrow h(k, i)$ 
4      if  $T[q] == \text{NIL}$  then
5           $T[q] \leftarrow k$ 
6          return  $q$ 
7      else
8           $i \leftarrow i + 1$ 
9  until  $i == m$ 
10 error "hash table overflow"
```

HASH-SEARCH(T, k)

```

1   $i \leftarrow 0$ 
2  repeat
3       $q \leftarrow h(k, i)$ 
4      if  $T[q] == k$  then
5          return  $q$ 
6       $i \leftarrow i + 1$ 
7  until  $T[q] == \text{NIL}$  or  $i == m$ 
8  return NIL
```

5.2 Linear Probing

Linear probing is the **simplest open-addressing approach** to resolving collisions. It is a **special case of double hashing** where the step size is fixed at 1.

Probe Function:

$$h(k, i) = (h'(k) + i) \bmod m$$

where $h'(k)$ is an ordinary hash function and $i = 0, 1, 2, \dots$ is the probe number.

How Linear Probing Works

1. An auxiliary hash function h' determines the first probe position $h'(k)$.
2. If slot $T[h'(k)]$ is occupied, probe the next position $T[h'(k) + 1]$.

3. Keep going as necessary, up to slot $T[m - 1]$.
4. **Wrap around** to slots $T[0]$, $T[1]$, and so on.
5. Never go past slot $T[h'(k) - 1]$ (ensures complete cycle without repetition).

Characteristics

Table 17: Linear Probing characteristics

Feature	Description
Probe sequence	$h'(k)$ alone determines the entire probe sequence.
Distinct probe sequences	Only m distinct probe sequences (one for each value of $h'(k)$).
Step size	Fixed at 1.

Primary Clustering

Primary Clustering Problem:

Linear probing suffers from **primary clustering** -- long runs of occupied slots build up, increasing average search time. Since all keys that hash to the same initial position follow the same probe sequence, collisions tend to cluster together.

Performance Analysis

For an unsuccessful search at load factor $\alpha = n/m$:

$$\text{Expected probes} \approx \frac{1}{2} \left(1 + \frac{1}{(1 - \alpha)^2} \right)$$

For a successful search:

$$\text{Expected probes} \approx \frac{1}{2} \left(1 + \frac{1}{1 - \alpha} \right)$$

Performance at different load factors:

At $\alpha = 0.5$: ~2.5 probes for unsuccessful search.

At $\alpha = 0.75$: ~8.5 probes.

At $\alpha = 0.9$: ~50 probes -- performance degrades rapidly as $\alpha \rightarrow 1$.

5.3 Quadratic Probing

Quadratic probing uses a quadratic function of the probe number to determine the next slot to examine, eliminating primary clustering.

Probe Function:

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$$

where c_1 and c_2 are constants (commonly $c_1 = 0$, $c_2 = 1$ or using triangular numbers).

Common simplification: Using $c_1 = c_2 = 1/2$ gives triangular number offsets:

$$h(k, i) = \left(h'(k) + \frac{i(i+1)}{2} \right) \bmod m$$

How Quadratic Probing Works

Quadratic Probing: $h(k,i) = (h'(k) + c_1i + c_2i^2) \bmod m$, with $c_1=0, c_2=1$

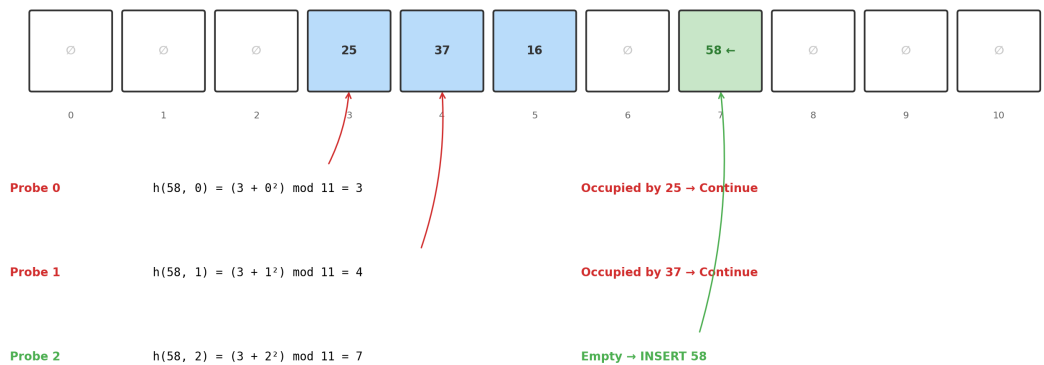


Figure 7: Quadratic Probing: inserting key 58 with $h'(58) = 3$. Probe 0: slot 3 (occupied); Probe 1: $(3 + 1) = 4$ (occupied); Probe 2: $(3 + 4) = 7$ (empty) -- insert 58. Quadratic gaps avoid primary clustering.

Characteristics

Table 18: Quadratic Probing characteristics

Feature	Description
Clustering	Avoids primary clustering because keys with the same initial hash follow different probe sequences.
Secondary Clustering	Still suffers from secondary clustering: if two keys have the same initial hash value, they follow the same probe sequence.
Slot coverage	If m is not chosen carefully, the probe sequence may not cover all slots. Must choose m to be prime and $c_2 < m$.

5.4 Double Hashing

Double hashing uses **two hash functions** to determine the probe sequence. The first gives the initial position; the second gives the step size.

Probe Function:

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

where $h_1(k)$ and $h_2(k)$ are two independent hash functions.

Requirements for Full Coverage

For the entire hash table to be searched, $h_2(k)$ **must be relatively prime to m** (i.e., $\gcd(h_2(k), m) = 1$).

Two convenient methods to ensure this:

Table 19: Ensuring full coverage in double hashing

Method	Description
Method 1	Let m be an exact power of 2, and design h_2 so that it always produces an odd number .
Method 2	Let m be prime , and design h_2 so that it always returns a positive integer less than m .

Practical Example

Given: $k = 123456$, $m = 701$ (prime), $m' = 700$.

Auxiliary hash functions:

$$h_1(k) = k \bmod m \quad h_2(k) = 1 + (k \bmod m')$$

Calculation:

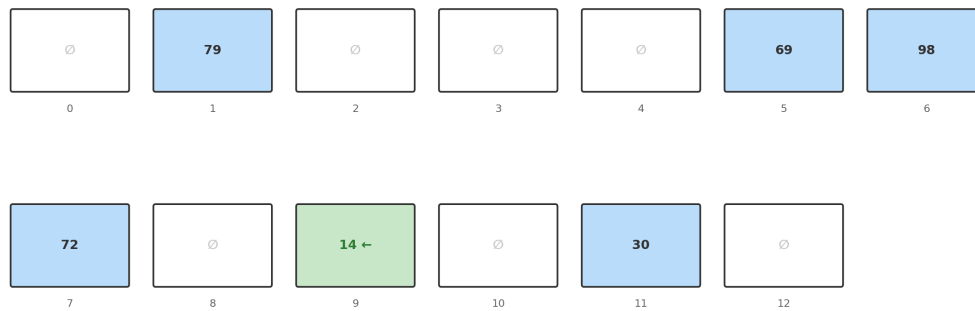
$$h_1(123456) = 123456 \bmod 701 = \mathbf{80}$$

$$h_2(123456) = 1 + (123456 \bmod 700) = 1 + 256 = \mathbf{257}$$

Probe sequence:

- First probe: position **80**
- Successive probes: examine every **257th slot** (modulo 701)
- Sequence: 80, $(80 + 257) \bmod 701 = 337$, $(337 + 257) \bmod 701 = 594, \dots$

Double Hashing: Inserting key 14, $h_1(k) = k \bmod 13$, $h_2(k) = 1 + (k \bmod 11)$



Probe 0 $h(14, 0) = (h_1(14) + 0 \cdot h_2(14)) \bmod 13 = (1 + 0) \bmod 13 = 1$

Slot 1: Occupied by 79 → Continue

Probe 1 $h(14, 1) = (1 + 1 \cdot 4) \bmod 13 = 5$

Slot 5: Occupied by 69 → Continue

Probe 2 $h(14, 2) = (1 + 2 \cdot 4) \bmod 13 = 9$

Slot 9: Empty → INSERT 14

Parameters:
 $h_1(k) = k \bmod 13$
 $h_2(k) = 1 + (k \bmod 11)$
 $k = 14: h_1(14) = 1, h_2(14) = 4$
 Probe sequence: $(1 + i \cdot 4) \bmod 13$
 $= \{1, 5, 9, 0, 4, 8, 12, 3, 7, 11, 2, 6, 10\}$

Figure 8: Double Hashing: inserting key 14 with $h_1(14) = 1$, $h_2(14) = 4$ into a table of size 13. Probe 0: slot 1 (occupied by 79); Probe 1: slot 5 (occupied by 69); Probe 2: slot 9 (empty) – insert 14.

Number of Probe Sequences

Theorem:

When m is **prime** or an **exact power of 2**, double hashing produces $\Theta(m^2)$ distinct probe sequences.

Reason: Each possible $(h_1(k), h_2(k))$ pair yields a **distinct probe sequence**. For such values of m , double hashing appears to perform close to the "ideal" scheme of independent uniform permutation hashing.

Proof: Double Hashing Covers All Slots

Theorem: If $\gcd(h_2(k), m) = 1$, then the probe sequence generates a permutation of $\{0, 1, \dots, m - 1\}$.

Proof: Suppose two probe numbers i and j (with $0 \leq i < j < m$) yield the same slot:

$$(h_1(k) + i \cdot h_2(k)) \equiv (h_1(k) + j \cdot h_2(k)) \pmod{m}$$

Subtracting $h_1(k)$ from both sides:

$$i \cdot h_2(k) \equiv j \cdot h_2(k) \pmod{m}$$

$$(j - i) \cdot h_2(k) \equiv 0 \pmod{m}$$

This means m divides $(j - i) \cdot h_2(k)$. But $0 < j - i < m$ and $\gcd(h_2(k), m) = 1$, so m cannot divide $(j - i)$ -- contradiction.

Therefore, all m probe positions generate distinct slots, forming a complete permutation.



5.5 Deletion in Open Addressing

Deletion is straightforward in chaining (remove from linked list) but **problematic in open addressing**.

The Problem with Direct Deletion

Consider: Key A was inserted at slot 5 after probing. Later, key B (which hashes to the same initial slot) was inserted at slot 6 after finding slot 5 occupied. If we delete A from slot 5, making it empty, a search for B would stop at slot 5 (which now appears empty) and **incorrectly conclude B is not in the table**.

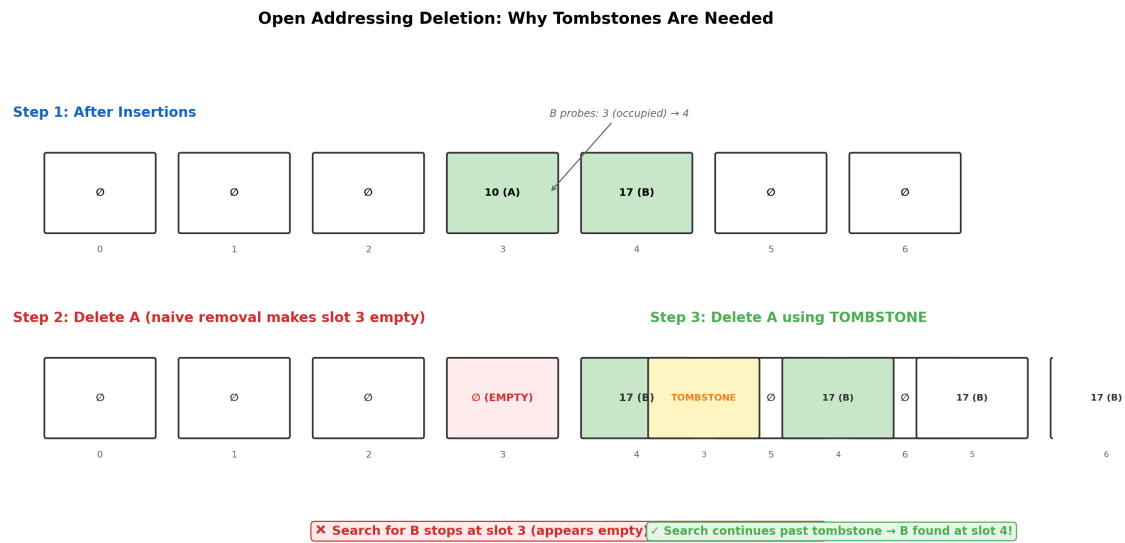


Figure 9: Why tombstones are needed: (top) after insertions; (middle) naive deletion of A makes slot 3 empty, breaking search for B; (bottom) tombstone preserves probe chain while marking slot as reusable for insertion.

Solution 1: Tombstones (Lazy Deletion)

Instead of clearing the slot, mark it with a special **tombstone** (or "deleted") marker.

Table 20: Tombstone behavior

Slot State	Search Behavior	Insertion Behavior
Occupied	Compare key; if match, found; else continue probing.	Continue probing.
Tombstone (Deleted)	Continue probing (key might be further ahead).	Insert here (slot reusable).
Empty (NIL)	Stop: key not present.	Insert here.

Solution 2: Rehashing

When the number of tombstones exceeds a threshold, **rebuild the entire hash table** by re-inserting all remaining keys into a fresh table. This removes all tombstones and restores optimal $O(1)$ expected performance.

Tombstone Accumulation Problem:

Over many deletions, tombstones can accumulate and degrade search performance because searches must traverse through them. Periodic rehashing is necessary when tombstones exceed a threshold (e.g., 1/4 of the table).

5.6 Comparison of Probing Methods

Table 21: Comparison of open addressing probing methods

Feature	Linear Probing	Quadratic Probing	Double Hashing
Formula	$(h'(k) + i) \bmod m$	$(h'(k) + c_1 i + c_2 i^2) \bmod m$	$(h_1(k) + i \cdot h_2(k)) \bmod m$
Step size	1 (fixed)	Quadratic in i	$h_2(k)$ (varies with key)
Probe sequences	m	m	$\Theta(m^2)$
m requirements	Any	Prime, $c_2 < m$	Prime or power of 2
Primary clustering	Yes (severe)	No	No
Secondary clustering	Yes	Yes	No
Cache locality	Excellent	Moderate	Poor
Expected probes (unsuccessful)	$\frac{1}{2}(1 + \frac{1}{(1-\alpha)^2})$	$\frac{1}{1-\alpha}$ (approx.)	$\frac{1}{1-\alpha}$
Expected probes (successful)	$\frac{1}{2}(1 + \frac{1}{1-\alpha})$	$-\frac{1}{\alpha} \ln(1-\alpha)$ (approx.)	$\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$

Part 6: Summary & Comparison Tables

6.1 Search Algorithms Comparison

Table 22: Linear Search vs. Binary Search

Feature	Linear Search	Binary Search
Prerequisite	None	Sorted array
Approach	Sequential	Divide & Conquer
Best Case	$\Theta(1)$	$\Theta(1)$
Average Case	$\Theta(n)$	$\Theta(\log n)$
Worst Case	$\Theta(n)$	$\Theta(\log n)$
Space	$O(1)$	$O(1)$ iterative / $O(\log n)$ recursive
In-Place	Yes	Yes
Adaptive	Partially	No
Data Structure	Any sequential	Random access (arrays)

6.2 Hashing Concepts Summary

Table 23: Key hashing concepts

Concept	Definition
Hash Table	Data structure storing elements in slots computed by hash function; size $m \ll U $.
Hash Function $h(k)$	Maps key k to slot index in $\{0, 1, \dots, m - 1\}$.
Collision	Two keys $k_1 \neq k_2$ where $h(k_1) = h(k_2)$.
Chaining	Collision resolution where each slot points to linked list of colliding elements.

Open Addressing	Collision resolution where all elements store in the table itself; no external storage.
Load Factor α	n/m (average elements per slot).
Independent Uniform Hashing	Each key equally likely to hash to any slot, independent of other keys.
Universal Hashing	Randomly selecting a hash function from a family to approximate SUHA.
Static Hashing	Single fixed hash function; not recommended for adversarial data.
Random Hashing	Hash function chosen randomly from family at runtime; recommended.
Primary Clustering	Long runs of occupied slots forming in linear probing.
Secondary Clustering	Keys with same initial hash following same probe sequence.
Tombstone	Special marker for deleted slots in open addressing to preserve probe chains.
Rehashing	Rebuilding the hash table to remove tombstones and restore performance.

6.3 Collision Resolution Comparison

Table 24: Collision resolution strategies: detailed comparison

Feature	Chaining	Linear Probing	Quadratic Probing	Double Hashing
Storage	External (linked lists)	Internal	Internal	Internal
Extra Space	$O(n)$ pointers	$O(1)$	$O(1)$	$O(1)$
α can exceed 1?	Yes	No	No	No
Deletion	Easy ($O(1)$ with doubly linked)	Tombstones needed	Tombstones needed	Tombstones needed
Clustering	None	Primary	Secondary	None
Expected search	$O(1 + \alpha)$	$O(\frac{1}{1-\alpha})$ (worse)	$O(\frac{1}{1-\alpha})$	$O(\frac{1}{1-\alpha})$

Cache performance	Poor	Excellent	Moderate	Poor
-------------------	------	-----------	----------	------

6.4 Hash Function Methods Comparison

Table 25: Hash function methods: comparison

Feature	Division	Multiplication	Multiply-Shift
Formula	$k \bmod m$	$\lfloor m(kA \bmod 1) \rfloor$	$(a \cdot k) \gg (w - p)$
Speed	Fast (1 division)	Moderate (mult + frac)	Very fast (mult + shift)
Table size m	Best: prime	Any value	Must be 2^p
Distribution quality	Moderate	Good	Good (2-independent)
Best use case	Simple implementations	Flexible table sizes	High-performance systems

Key Takeaways:

- **Linear Search:** Simple, no prerequisites, $O(n)$ -- use for small or unsorted data.
- **Binary Search:** Requires sorted data, $O(\log n)$ -- use for large sorted arrays.
- **Hash Tables:** Average $O(1)$ for all operations -- best for frequent lookups with key-based access.
- **Chaining:** Simple, handles overflow, but uses extra space for pointers.
- **Open Addressing:** No extra space, but needs careful probing and tombstones for deletion.
- **Double Hashing:** Best probing method -- eliminates clustering with $\Theta(m^2)$ probe sequences.